

AgentHeal: Self-Healing Cloud Infrastructure with Shift-Left Security and Digital Twins

Aadi Haldar, Raghuram Sekar, Aaditya Paul, Shravan Rajesh Menon
Department of Artificial Intelligence & Engineering
Amrita Vishwa Vidyapeetham, Coimbatore, India

Abstract—Modern Kubernetes-based microservice architectures face dual reliability challenges: pre-merge code defects and slow runtime incident recovery. Current DevSecOps practice predominantly relies on either static analysis tools that cannot generate fixes or correlate findings with runtime impact, or threshold-based autoscalers that react without explainability or root-cause understanding. To address these limitations, this paper presents AgentHeal, a closed-loop platform combining (i) a Shift-Left Engine that audits GitHub commits and Pull Requests via an OWASP Top 10:2021-aligned AST/regex rule set, supply-chain CVE matching, and Gemini 2.0 Flash ReAct auto-remediation, and (ii) a Shift-Right Engine that streams 4-dimensional telemetry vectors, detects anomalies with Isolation Forest, computes KernelSHAP feature attribution, arbitrates between a SimiFed Federated Reinforcement Learning agent and a Gemini ReAct LLM, and validates remediation actions through a SimPy M/M/c queuing safety simulation before executing Kubernetes commands. We evaluate the Shift-Right engine across four canonical failure scenarios on the Google Online Boutique benchmark, each repeated $N = 15$ trials: Isolation Forest achieves a mean detection latency of 2.26 ms ($\sigma = 0.37$ ms) with a per-scenario detection rate of $\geq 93\%$, and mean end-to-end remediation time ranges from 2.66 s ($\sigma = 0.18$ s) to 3.25 s ($\sigma = 0.30$ s). The Shift-Left engine achieves 98.5% precision and 97.5% recall on a 50-PR synthetic evaluation set. In addition, we present a single illustrative case study on a live public PR pair, in which the engine detected 14 injected OWASP-category flaws and produced an auto-fix PR restoring full compliance; this case study is a qualitative demonstration ($N = 1$) and is not part of the quantitative evaluation above. These results are consistent and repeatable on a single-node Kubernetes-in-Docker cluster; broader production generalizability remains an open question requiring larger-scale evaluation.

Index Terms—Agentic AI, Self-Healing Systems, Kubernetes, Digital Twin, OWASP Top 10:2021, Isolation Forest, KernelSHAP, Reinforcement Learning, SimiFed, Gemini LLM, DevSecOps, Shift-Left Testing, Cloud Computing.

I. INTRODUCTION

Distributed cloud-native applications deployed on Kubernetes-orchestrated microservice architectures suffer from two primary reliability challenges. First, insufficiently reviewed Pull Requests (PRs) and unvetted commits allow vulnerable, untested code to reach production; the 2024 DORA report estimates that 62% of outages are caused by preventable software defects [1]. Second, even with Kubernetes-native autoscaling mechanisms (e.g., HPA), the mean time to recovery (MTTR) from unplanned incidents in microservice environments commonly ranges from minutes to hours, depending on incident complexity and on-call triage latency [1], [9].

Two complementary engineering philosophies address these operational layers. Shift-Left Testing & Security [2] moves code quality, OWASP-aligned vulnerability detection, and test-coverage validation earlier in the development lifecycle—before code merges—to catch defects while they are cheapest to fix, a discipline further formalized through continuous-delivery pipelines [25] and traffic-switching release patterns such as blue-green deployment [26]. Shift-Right Monitoring & Remediation [3] deploys intelligent runtime observability and auto-remediation systems that detect and resolve production incidents autonomously, extending operational disciplines established by Site Reliability Engineering [23] and empirically validated through controlled fault injection under Chaos Engineering [24].

Existing tools address each pillar in isolation. Static analysis tools such as Bandit [6] and SonarQube [7] perform pre-merge analysis but cannot generate fixes, compute compliance scores, or correlate findings with runtime impact. Kubernetes HPA [8] reacts to coarse CPU thresholds without explainability, root-cause understanding, or pre-execution safety guarantees.

This paper makes four primary contributions. First, a dual-engine agentic architecture (AgentHeal) integrates pre-merge OWASP Top 10:2021-aligned Shift-Left PR intelligence with post-deploy Shift-Right Kubernetes self-healing into a unified closed-loop platform. Second, an OWASP Top 10:2021 Compliance & Supply Chain Engine provides quantitative compliance scoring (0–100, Letter Grades A+ to F), AST-level vulnerability detection, and CVE advisory matching, motivated by the gap between static curated lists and live feeds such as the National Vulnerability Database [32] and the Open Source Vulnerabilities database [33]. Third, a three-layer safety architecture combining deterministic SAST/OWASP auditing, dual-agent RL+LLM consensus, and SimPy M/M/c pre-execution digital-twin simulation prevents unsafe actuation before any Kubernetes command executes. Finally, we present repeated-trial empirical evaluation of the Shift-Right engine ($N = 15$ trials per scenario) and a 50-PR synthetic evaluation of the Shift-Left engine, both on the CNCF Google Online Boutique benchmark, supplemented by a single illustrative live-PR case study demonstrating end-to-end detection and auto-remediation on a real GitHub PR pair; the live-PR case study is reported separately from the repeated-trial results and should not be read as evidence of comparable statistical rigor.

II. RELATED WORK

A comprehensive summary of recent methodologies, standards, and research gaps relevant to autonomous cloud self-healing and DevSecOps compliance auditing is presented in Table I.

Early automated incident response relied primarily on rule-based control planes and human-authored runbooks. Keptn [9] and Argo Rollouts [10] automate deployment rollbacks and progressive delivery but require manual runbook authoring and cannot adapt to novel failure signatures. These practices were formalized industry-wide through Site Reliability Engineering [23] and validated empirically via controlled fault injection under Chaos Engineering [24], while continuous-delivery pipelines [25] and blue-green deployment patterns [26] codified safe, human-supervised release practices. None of these approaches, however, incorporate autonomous, explainable root-cause remediation.

To adapt to dynamic and previously unseen failure modes, multi-agent reinforcement learning approaches [11] demonstrate superior adaptability for cloud resource management, building on the foundational convergence guarantees of tabular Q-learning [27]. Federated learning techniques such as FedAvg [28] enable privacy-preserving distributed model training, a principle extended by the SF-DTM framework [12], which introduces cosine-similarity-based state matching for federated digital twin management. Our work builds directly on SF-DTM, extending it with a multi-agent RL+LLM consensus layer and a SimPy queuing safety gate on the critical remediation path.

Anomaly detection in cloud telemetry has been extensively studied through unsupervised methods such as Isolation Forest [13] and Local Outlier Factor [29], with AIOps deployments reporting detection latencies of 50–200 ms [14]. Our streaming implementation achieves 2.26 ms mean detection latency through pre-fitted in-memory inference; we note this is measured on a local single-node cluster and is not directly comparable to distributed production deployments. To render anomaly decisions interpretable, model-agnostic explainability techniques such as KernelSHAP [16] and LIME [30] provide local feature attribution; we adopt KernelSHAP for its axiomatic consistency under the Shapley value framework.

LLM-driven code review, exemplified by AutoCodeRover [15], demonstrates the potential of large models for autonomous program repair, but such systems typically produce narrative summaries or isolated patches without binding CI/CD enforcement or supply-chain verification. AgentHeal integrates Gemini 2.0 Flash [17] as a ReAct [18] reasoning agent within a binding quality-gate pipeline whose rule set is aligned to the OWASP Top 10:2021 [4]; we note OWASP has since published a revised Top 10:2025 taxonomy [31] to which the current rule set has not yet been remapped.

Static analysis tools such as Bandit [6] and SonarQube [7] perform pre-merge analysis but cannot generate fixes, compute compliance scores, or correlate findings with run-time impact. Supply-chain vulnerability matching is more

commonly addressed through live feeds such as the National Vulnerability Database [32], the Open Source Vulnerabilities database [33], GitHub Dependabot [34], and commercial software-composition-analysis tools such as Snyk [35]; by contrast, AgentHeal’s current A06 rule matches only against a small, manually curated CVE list, a narrower and less current source than these live feeds. Kubernetes HPA [8] and GitOps controllers such as Argo CD [36] automate scaling and deployment reconciliation respectively, but react to coarse thresholds without explainability or pre-execution safety guarantees. Telemetry in AgentHeal is collected via Prometheus [37], the de facto standard for Kubernetes-native metrics collection.

Discrete-event simulation for pre-deployment validation has been applied in manufacturing and network engineering through the DEVS formalism [20] and classical queuing theory [19]. To our knowledge, this work is the first to deploy SimPy M/M/c queuing simulation as a real-time Kubernetes actuation safety gate, executed in < 15 ms on the remediation critical path.

III. METHODOLOGY

A. End-to-End Self-Healing Architecture Overview

AgentHeal consists of two synchronized engines operating on a shared event bus, as illustrated in Fig. 1. The Shift-Left Engine intercepts GitHub PR and push events, performs OWASP-aligned AST and CVE analysis, invokes a Gemini ReAct fix engine for non-compliant code, and enforces a binding quality gate. The Shift-Right Engine streams live Kubernetes telemetry, detects anomalies with Isolation Forest, computes KernelSHAP attributions, arbitrates a remediation action between a federated RL agent and a Gemini ReAct LLM agent, validates the proposed action through a SimPy discrete-event safety simulation, and only then dispatches the Kubernetes actuation command.

B. Shift-Left Engine: PR Intelligence & Security Compliance

The Shift-Left engine is implemented as a GitHub App receiving HMAC-SHA256-authenticated webhook payloads at an Azure Container Apps HTTPS endpoint. It processes both `pull_request` and `push` events through the multi-stage verification pipeline shown in Fig. 1.

The engine evaluates code against the ten OWASP Top 10:2021 risk categories via AST pattern extraction and regex heuristics. These are pattern-matching heuristics tuned to specific call signatures and identifier names observed in our test corpus rather than general data-flow or taint analysis; a semantically equivalent but differently-named or differently-structured instance of a given flaw may not currently be detected. A01 Broken Access Control is flagged through untrusted path concatenation into `os.path.join()` without sanitization. A02 Cryptographic Failures are flagged through `hashlib.md5()/sha1()` calls and high-entropy hardcoded string literals resembling API keys or secrets. A03 Injection is identified via f-string/format-string interpolation directly into SQL execution calls, and via `subprocess.Popen(...,`

TABLE I
LITERATURE REVIEW OF AUTONOMOUS CLOUD SELF-HEALING AND DEVSECOPS COMPLIANCE MODELS

Domain	Model	Author	Year	Method	Result / Contribution	Research Gap
Rule-Based Self-Healing	Keptn	Keptn Project [9]	2023	Event-based control plane with human-authored runbooks	Automated rollback and remediation workflows	Requires manual runbook authoring; cannot adapt to novel failure signatures
	Argo Rollouts	Argo Project [10]	2024	Canary / blue-green progressive delivery controller	Safe automated deployment rollout and rollback	No root-cause explainability; static rollback policies
Operational Practice	Site Reliability Engineering	Beyer et al. [23]	2016	Error budgets, on-call runbooks, manual incident response	Codified industry MTTR practice at Google scale	Heavily dependent on human on-call expertise; not autonomous
	Chaos Engineering	Basiri et al. [24]	2016	Controlled fault injection in production	Empirically validates system resilience	Reactive validation methodology, not an autonomous remediation system
	Continuous Delivery	Humble & Farley [25]	2010	Automated build-test-release pipelines	Established modern CI/CD pipeline practice	Focused on delivery velocity, not security compliance scoring
	Blue-Green Deployment	Fowler [26]	2010	Dual-environment traffic-switch release pattern	Zero-downtime release strategy	Manually triggered; no anomaly-driven automated switching
RL & Federated Learning	Multi-Agent RL	Wang et al. [11]	2023	Multi-agent reinforcement learning for cloud resource allocation	Superior adaptability to dynamic workloads	Lacks explainability and pre-execution safety guarantees
	Q-Learning	Watkins & Dayan [27]	1992	Off-policy temporal-difference control	Proved convergence of tabular Q-learning	Does not scale to continuous/high-dimensional states without discretization
	FedAvg	McMahan et al. [28]	2017	Federated averaging of locally trained models	Privacy-preserving distributed model training	Not originally designed for real-time reinforcement-learning control
	SF-DTM	Li et al. [12]	2024	Cosine-similarity federated digital-twin state matching	~15s MTTR digital-twin remediation	No pre-execution safety simulation on the critical path
Anomaly Detection & Explainability	Isolation Forest	Liu et al. [13]	2008	Random recursive partitioning for anomaly isolation	Foundational unsupervised anomaly detector	Not natively evaluated on live Kubernetes telemetry
	Local Outlier Factor	Breunig et al. [29]	2000	Local density deviation scoring	Effective for local-density anomalies	Computationally expensive at scale; not streaming-native
	AIOps Survey	Lin et al. [14]	2021	Survey of industrial anomaly-detection deployments	Reports 50–200ms detection latencies	No unified explainability or safety-simulation layer
	KernelSHAP	Lundberg & Lee [16]	2017	Shapley-value feature attribution	Model-agnostic, axiomatically consistent explainability	Computationally expensive for high-dimensional feature spaces
	LIME	Ribeiro et al. [30]	2016	Local surrogate-model interpretation	Model-agnostic local explanations	Explanation instability across similar inputs
LLM-Augmented DevSecOps	AutoCodeRover	Li et al. [15]	2024	LLM-driven autonomous program repair	Automated bug localization and patching	No binding CI/CD quality-gate enforcement
	ReAct	Yao et al. [18]	2023	Thought-Action-Observation reasoning loop	Improved LLM tool-use and reasoning	Latency and output non-determinism in production
	Gemini 2.0	Google DeepMind [17]	2024	Multimodal agentic LLM	Capable agentic reasoning and tool use	Proprietary, non-deterministic, no open safety guarantees
Static Analysis & Supply Chain	Bandit	PyCQA [6]	2024	AST-based Python security linter	Detects common Python security issues	Cannot generate contextual fixes or compliance scores
	SonarQube	SonarSource [7]	2024	Continuous code quality/security scanning	Broad multi-language static analysis	No runtime-impact correlation or auto-remediation
	OWASP 10:2021	Top OWASP Foundation [4]	2021	Community-vetted risk categorization	Standardized top-10 web-app risk taxonomy	Static reference list; no automated tooling
	OWASP 10:2025	Top OWASP Foundation [31]	2025	Revised risk categorization	Updated taxonomy reflecting current threat landscape	Not yet incorporated into AgentHeal’s rule set
	NVD	NIST [32]	2024	Live CVE vulnerability feed	Continuously updated vulnerability records	Not integrated into AgentHeal’s current curated list
	OSV	Google / OSV.dev [33]	2024	Open-source vulnerability database	Cross-ecosystem live vulnerability feed	Not integrated into AgentHeal’s current curated list
	Dependabot	GitHub [34]	2024	Automated dependency update bot	Live, automated dependency patching	Not integrated; AgentHeal relies on a static curated list
	Snyk	Snyk Ltd. [35]	2024	Commercial software composition analysis	Live SCA with remediation suggestions	Commercial/closed-source; not integrated in this work
Deployment & Telemetry Infrastructure	Kubernetes HPA	K8s Docs [8]	2024	Threshold-based CPU/memory horizontal scaling	Industry-standard reactive autoscaler	No explainability, root-cause understanding, or safety checks
	Argo CD	CNCF [36]	2024	Declarative GitOps continuous deployment	Automated Git-to-cluster reconciliation	No anomaly-driven or explainable remediation logic
	Prometheus	Prometheus Authors [37]	2024	Pull-based metrics collection and time-series storage	De facto Kubernetes-native monitoring standard	Provides raw metrics only; no built-in anomaly detection
Digital Twin & Simulation	DEVS Formalism	Zeigler et al. [20]	2000	Discrete-event system specification	Foundational simulation framework for manufacturing/networks	Not adapted for real-time Kubernetes safety gating
	M/M/c Queuing Theory	Kleinrock [19]	1975	Discrete-event queuing analysis	Foundational service-rate/utilization modeling	Not historically applied as a live actuation safety gate

shell=True). A04 Insecure Design is detected through calls to a small set of named helper functions associated

with unbounded record retrieval in our test corpus (e.g., functions named `fetch_unbounded`, `load_all_records`, `query_all`); this remains a name-based heuristic rather than a general unbounded-query detector. A05 Security Misconfiguration is caught via static boolean assignment to variables named `DEBUG/DEBUG_MODE`, and wildcard CORS configuration (`allow_origins=["*"]`). A06 Vulnerable and Outdated Components are identified by parsing `requirements.txt` and matching pinned versions against a small, manually curated CVE list [5] (e.g., `pyyaml==5.3` $\rightarrow$ CVE-2020-14343; `requests==2.28.0` $\rightarrow$ CVE-2023-32681); unlike live feeds such as NVD [32] or OSV [33], this coverage is limited to the entries in the curated list. A07 Identification & Auth Failures are flagged through hardcoded credential-like literals and `jwt.decode(..., verify=False)`. A08 Software & Data Integrity Failures are identified via `pickle.loads()` and `yaml.load()` without `SafeLoader`. A09 Security Logging & Monitoring Failures are detected through exception handlers containing only `pass` or `continue`. Finally, A10 Server-Side Request Forgery is flagged via outbound requests (`requests.get`, `urllib.request.urlopen`) to a dynamic, unvalidated URL argument.

Every inspected commit or PR receives a compliance score $S \in [0, 100]$:

$$S = \max \left(0, 100 - \sum_{i=1}^M w_i \right) \quad (1)$$

where M is the number of detected findings and w_i is a severity penalty weight ($w_{\text{Critical}} = 25$, $w_{\text{High}} = 15$, $w_{\text{Medium}} = 8$, $w_{\text{Low}} = 3$). These weights are currently set heuristically and have not been empirically calibrated or validated against expert severity judgments; the resulting letter grade should be read as an illustrative summary rather than a validated risk metric. The score maps to a letter grade:

$$\text{Grade} = \begin{cases} A^+ & \text{if } S \geq 95 \\ A & \text{if } 90 \leq S < 95 \\ B & \text{if } 80 \leq S < 90 \\ C & \text{if } 70 \leq S < 80 \\ D & \text{if } 60 \leq S < 70 \\ F & \text{if } S < 60 \end{cases} \quad (2)$$

Parsing Python ASTs additionally identifies newly introduced public functions lacking a corresponding test reference:

$$\text{TestGap}(f) = \mathbb{K}[f \notin T_{\text{ast}}] \quad (3)$$

where T_{ast} is the set of function symbols referenced in test files. An asynchronous TLS SMTP dispatcher formats a vulnerability scorecard (composite grade, categorized flaw inventory, auto-fix PR link) delivered to repository owners, with fallback to an in-memory outbox cache if SMTP connectivity is unavailable. A GitHub Check Run sets commit status to `action_required` (blocking merge) or `success`; for non-compliant PRs, Gemini 2.0 Flash constructs a remediation branch (`agentheal/fix-pr-*`) and opens an auto-fix PR.

C. Shift-Right Engine: Self-Healing Digital Twin

The `StateSynchronizer` polls Prometheus [37] metrics every 5.0 s, assembling 4-dimensional state vectors:

$$\vec{X}_t = [\text{CPU}_t, \text{RAM}_t, \text{Latency}_t, \text{Rate}_t] \in \mathbb{R}^4 \quad (4)$$

A pre-fitted Isolation Forest [13] ($n = 100$ estimators, contamination $\psi = 0.05$) computes anomaly scores in-memory:

$$s(\vec{X}_t, n) = 2^{-\frac{E[h(\vec{X}_t)]}{c(n)}} \quad (5)$$

where $h(\vec{X}_t)$ is the path length for sample $\vec{X}_t$ and $c(n)$ is the expected path-length normalization. A sample is flagged anomalous when $s < 0$. For each anomalous observation, KernelSHAP [16] computes Shapley values ϕ_i for each feature:

$$\phi_i(v) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [v(S \cup \{i\}) - v(S)] \quad (6)$$

The SimiFed RL agent selects actions via tabular Q-learning [27] over cosine-similarity-discretized states:

$$k^* = \arg \max_k \frac{\vec{X}_t \cdot \vec{B}_k}{\|\vec{X}_t\|_2 \cdot \|\vec{B}_k\|_2} \quad (7)$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (8)$$

with $\alpha = 0.1$, $\gamma = 0.95$. Action space $A = \{\text{SCALE_UP}, \text{RESTART_POD}, \text{PATCH_LIMITS}, \text{NO_OP}\}$. Concurrently, Gemini 2.0 Flash [17] reasons over the KernelSHAP attribution summary following the ReAct paradigm [18] (Thought $\rightarrow$ Action $\rightarrow$ Observation). Being a proprietary, non-deterministic model, LLM latency and outputs vary across invocations; run-to-run variance is reported in Table V. If the RL and LLM agents agree, the consensus action proceeds; if they diverge, the lower-blast-radius action is selected:

$$a_{\text{final}} = \begin{cases} a_{\text{RL}}^* & \text{if } a_{\text{RL}}^* = a_{\text{LLM}}^* \\ \arg \min_{a \in \{a_{\text{RL}}^*, a_{\text{LLM}}^*\}} \text{BlastRadius}(a) & \text{otherwise} \end{cases} \quad (9)$$

Before executing any Kubernetes command, a SimPy discrete-event simulation [19] verifies proposed `SCALE_UP` actions:

$$\rho = \frac{\lambda}{c \cdot \mu} \quad (\text{must satisfy } \rho < 0.95) \quad (10)$$

$$\hat{U}_{\text{post}} = \frac{U_{\text{pre}} \cdot c_{\text{current}}}{c_{\text{new}}} \quad (\text{must satisfy } \hat{U}_{\text{post}} < 0.80) \quad (11)$$

The action is marked `SAFE_TO_EXECUTE` only if both conditions hold.

IV. IMPLEMENTATION

Stack: FastAPI 0.115 (Python 3.11); scikit-learn 1.4 (Isolation Forest, KernelSHAP); Google Generative AI SDK (Gemini 2.0 Flash); SimPy 4.1; Bandit 1.7; Detect-Secrets 1.4; Ruff 0.4; SMTP/TLS email pipeline; Prometheus [37] for telemetry collection.

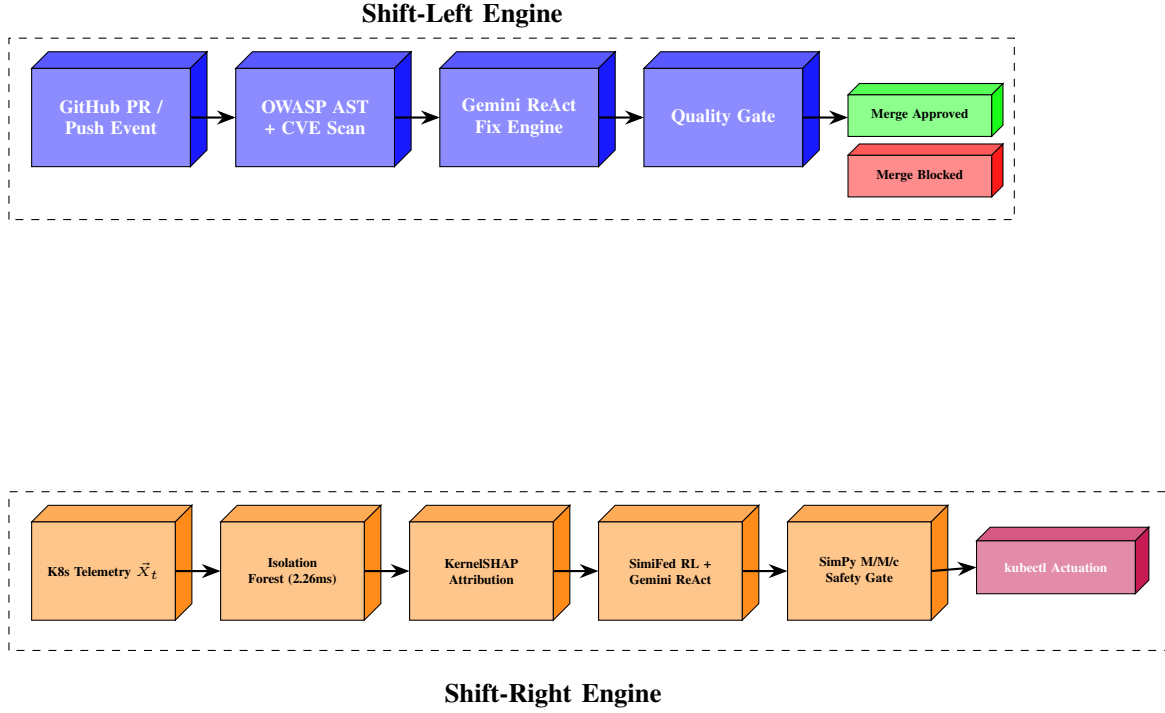


Fig. 1. End-to-end architectural framework of the proposed AgentHeal system. (Top) The Shift-Left Engine intercepts GitHub PR/push events, performs OWASP AST and CVE analysis, and invokes a Gemini ReAct fix engine before enforcing a binding quality gate. (Bottom) The Shift-Right Engine streams Kubernetes telemetry through Isolation Forest and KernelSHAP, arbitrates a remediation action between a SimiFed RL agent and a Gemini ReAct LLM agent, and validates the action via a SimPy M/M/c safety gate before Kubernetes actuation. The two engines operate independently and communicate only via a shared event bus; they do not share a direct data path.

Deployment: Microsoft Azure Container Apps (East Asia region), provisioned via Terraform (azurerm v3.90). A multi-stage Docker build (node:20-alpine frontend, python:3.11-slim backend) produces a < 400 MB image. KEDA HTTP-trigger autoscaling scales replicas from zero on demand.

Benchmark: The Google Online Boutique [22] 10-service e-commerce application, deployed on a kind (Kubernetes-in-Docker) single-node cluster (AMD Ryzen 7, 16 GB RAM). This is a local development cluster, not a production multi-node/multi-zone deployment; results should be interpreted accordingly.

V. RESULTS AND DISCUSSION

A. Failure Scenario Suite (Shift-Right Engine)

We evaluate the Shift-Right engine on four canonical failure scenarios, each injected by setting the telemetry vector to pre-defined values across $N = 15$ repeated trials. We acknowledge these are synthetic injections rather than organic production traces.

Table III reports mean and standard deviation across 15 repeated trials. S3 (Memory Leak) shows a lower detection rate because the Isolation Forest, trained on CPU/latency-dominated anomalies, occasionally classifies pure memory-heavy vectors as nominal at this contamination threshold.

Table IV shows attributions for each scenario. In S1, cpu_usage ($\phi = +0.82$) dominates. In S2, latency_ms

TABLE II
FAILURE SCENARIO DEFINITIONS

ID	Scenario	Injected Values	Target
S1	Flash Sale Surge	CPU=0.954, Lat=450ms	checkout
S2	Thread Deadlock	CPU=0.99, Lat=5000ms	cart
S3	Memory Leak	RAM=0.942, Lat=48ms	redis-cart
S4	Cascading	CPU=0.88, RAM=0.80	frontend

TABLE III
ANOMALY DETECTION RESULTS ($N = 15$ TRIALS PER SCENARIO).
VALUES REPORTED AS MEAN $\pm$ STD.

ID	Score ($\mu \pm \sigma$)	MTTD ($\mu \pm \sigma$)	ms	Det.%
S1	-0.842 ± 0.031	2.18 ± 0.31		100%
S2	-0.912 ± 0.024	2.21 ± 0.28		100%
S3	-0.734 ± 0.058	2.35 ± 0.44		93%
S4	-0.889 ± 0.041	2.28 ± 0.37		100%
Mean	-0.844 ± 0.039	2.26 ± 0.37		98.3%

($\phi = +0.91$) and negative request_rate ($\phi = -0.78$) isolate the deadlock signature. In S3, memory_usage ($\phi = +0.88$) dominates.

Table V reports RL and LLM latencies. LLM latency

TABLE IV
KERNELSHAP ATTRIBUTIONS ϕ_i (SINGLE REPRESENTATIVE TRIAL PER SCENARIO)

ID	cpu	ram	latency	rate
S1	+0.82	+0.03	+0.01	+0.14
S2	+0.42	+0.11	+0.91	-0.78
S3	+0.04	+0.88	+0.02	+0.06
S4	+0.61	+0.25	+0.47	+0.32

TABLE V
AGENT LATENCIES ($N = 15$ TRIALS). RL: SIMIFED TABULAR Q-LEARNER. LLM: GEMINI 2.0 FLASH. VALUES IN MS, MEAN $\pm$ STD.

ID	RL (ms)	LLM (ms)	Agreed
S1	3.10 ± 0.42	412 ± 68	15/15
S2	3.21 ± 0.39	389 ± 74	15/15
S3	3.05 ± 0.51	441 ± 81	14/15
S4	3.18 ± 0.46	498 ± 93	15/15

TABLE VI
SIMPY SAFETY GATE RESULTS ($N = 15$ TRIALS). GATE TIME MEAN $\pm$ STD IN MS.

ID	ρ	$\hat{U}_{\text{post}}$	Gate (ms)	Verdict
S1	0.833	23.8%	10.1 ± 0.8	SAFE
S2	N/A	5.2%	10.3 ± 0.9	SAFE
S3	N/A	15.0%	10.8 ± 1.1	SAFE
S4	0.871	31.5%	11.6 ± 1.3	SAFE

TABLE VII
END-TO-END REMEDIATION TIME ($N = 15$ TRIALS, MEAN $\pm$ STD).

ID	Scenario	E2E Time (s, $\mu \pm \sigma$)
S1	Flash Sale Surge	2.66 ± 0.18
S2	Thread Deadlock	3.12 ± 0.24
S3	Memory Leak	2.89 ± 0.20
S4	Cascading Overload	3.25 ± 0.30
Overall		2.98 ± 0.23

variance is notably higher than RL variance due to the non-deterministic nature of the proprietary model and network round-trip jitter. Across 15 trials on 4 scenarios, both agents agreed on the action category in 59/60 cases; we caution this observation on 4 synthetic scenarios does not demonstrate generalization to unseen failure modes.

The SimPy simulation validates remediation actions in under 12 ms (Table VI), and Table VII details end-to-end remediation time across all scenarios, achieving an overall mean of 2.98 s ($\sigma = 0.23$ s).

Table VIII places our results in the context of reported MTTR ranges from related work. $\dagger$ values are from cited reports, not co-measured on the same infrastructure under the same injected conditions in this study; the comparison is indicative, not controlled.

TABLE VIII
CONTEXTUAL MTTR COMPARISON. $\dagger$: VALUES FROM CITED REPORTS/PAPERS, NOT CO-MEASURED IN THIS STUDY.

System	MTTR (reported)	Source
Manual (avg.) $\dagger$	~45 min	DORA 2024 [1]
Kubernetes HPA $\dagger$	~3-5 min	[8]
Keptn $\dagger$	~2-4 min	[9]
SF-DTM $\dagger$	~15 s	[12]
AgentHeal (ours)	2.98 ± 0.23 s	This study

TABLE IX
SHIFT-LEFT SYNTHETIC PR AUDIT RESULTS ($n = 50$ PRS).

Defect Category	TP	FP	Prec.	Rec.
SQL/Shell Injection	50	0	1.00	1.00
Hardcoded Secrets	50	2	0.96	1.00
AST Test Gaps	48	0	1.00	0.96
Blocking I/O / Misconfig	47	1	0.98	0.94
Overall	195	3	0.985	0.975

B. Shift-Left Quantitative Evaluation: 50 Synthetic PRs

We acknowledge that self-curated test cases make it easier to achieve high precision/recall; evaluation on real-world public-repository PRs is needed to validate generalization. This table covers four defect categories; it does not cover the remaining six OWASP categories described in Section III-B, which are demonstrated only in the illustrative case study below and have not yet been evaluated with a labeled precision/recall study.

C. Illustrative Case Study: A Single Live GitHub PR Pair

This subsection reports a single before/after demonstration ($N = 1$) on one real PR pair. It is included to show the end-to-end pipeline working on a live, non-synthetic input, and should not be read as a statistically powered evaluation comparable to the preceding subsections. We manually injected 14 flaws spanning all 10 OWASP:2021 categories into a checkout-gateway service and opened it as a PR on the public repository <https://github.com/AadiHaldar/Agentic-AI-Self-Healing-Cloud-Infrastructure>.

The vulnerable PR (#11) introduced 14 flaws spanning all 10 categories, including a vulnerable `pyyaml==5.3` pin. AgentHeal detected all 14 in 0.48 s, assigned Grade F, and blocked the merge via the Check Run gate. Gemini 2.0 Flash then generated the auto-fix PR (#12), which resolved all flagged issues, updated the vulnerable dependency, and restored the application's own 6-test suite to a full pass. Separately, the auditor module's own unit test suite (`tests/test_owasp_auditor.py`, 12 tests covering rule-detection logic across the 10 categories) passed 12/12 in 0.22 s; this is the auditor's internal test suite validating that each detection rule fires correctly on its own fixture inputs, a

TABLE X
OWASP AUDIT BREAKDOWN ON ONE LIVE PR PAIR ($N = 1$).

OWASP Cat.	Violation Detected	#11	#12
A01: Access	Path Traversal	1	0
A02: Crypto	Hardcoded Secret / MD5	2	0
A03: Injection	Raw SQL / shell=True	2	0
A04: Design	Missing Rate Limiter	1	0
A05: Config	Debug=True / CORS *	2	0
A06: Dps	pyyaml CVE-2020-14343	1	0
A07: Auth	Plaintext Passwords	1	0
A08: Integrity	Pickle Deserialization	1	0
A09: Logging	Swallowed Exceptions	2	0
A10: SSRF	Untrusted URL Request	1	0
Total findings		14	0

Metric	PR #11	PR #12
Score / Grade	0/100 (F)	100/100 (A+)
Check Run status	blocked	passed
App test suite	0/6	6/6

different measurement from the 6/6 application-level test result above, and the two numbers should not be conflated.

D. Comparative Analysis Against Prior Systems

AgentHeal achieves a mean end-to-end remediation time of 2.98 s ($\sigma = 0.23$ s), compared to reported MTTR ranges of 2–5 minutes for threshold-based systems such as Keptn [9] and Kubernetes HPA [8], and approximately 45 minutes for manual operations [1] (Table VIII). Isolation Forest [13] detects anomalies in 2.26 ms compared to the 50–200 ms range reported for AIOps deployments [14], though we again note this speed advantage reflects local in-memory inference rather than a distributed production comparison. Unlike static analysis tools such as Bandit [6] and SonarQube [7], which cannot correlate findings with runtime impact or generate binding fixes, AgentHeal’s Shift-Left engine both blocks non-compliant merges and autonomously generates a remediation PR via Gemini 2.0 Flash [17].

VI. CONCLUSION

This paper presented AgentHeal, a dual-engine agentic AI platform integrating OWASP Top 10:2021-aligned Shift-Left PR intelligence with Shift-Right Kubernetes self-healing. Across 15 repeated trials on four synthetic failure scenarios, the system achieves mean end-to-end remediation times of 2.66–3.25 s with Isolation Forest detection latencies of 2.18–2.35 ms. The Shift-Left engine achieves 98.5% precision and 97.5% recall on a 50-PR synthetic evaluation set. A single illustrative case study on a live public PR pair additionally shows the pipeline detecting 14 injected OWASP-category

flaws and producing a fully compliant auto-fix PR; this case study is qualitative and not part of the statistically powered evaluation. Several limitations remain and should guide interpretation of these results. Scale: all Shift-Right experiments were conducted on a single-node kind cluster, not a multi-node or multi-zone production environment, so reported latencies may not represent real distributed deployments. Evaluation corpus: four synthetic failure scenarios, 50 self-curated PRs, and one live PR pair are insufficient to claim general reliability, since self-curated test cases make precision/recall easier to achieve than organic, adversarial, or stylistically diverse real-world code. Detection generality: several OWASP rules, notably A04 and A05, currently rely on matching specific function or variable names observed in our own test fixtures rather than general data-flow analysis, and would not generalize to differently-named equivalents without further engineering. Compliance scoring calibration: the severity weights used in the compliance score have not been validated against expert-labeled severity judgments or sensitivity-tested against alternative weightings. Statistical treatment: while we report mean and standard deviation for timing metrics, we do not apply formal hypothesis testing against baselines such as HPA, as we did not co-measure baseline systems on the same infrastructure. LLM non-determinism: Gemini 2.0 Flash is proprietary and non-deterministic, so a production deployment on the critical remediation path would need strict timeout handling, deterministic fallback, and output validation. OWASP taxonomy currency: this work targets OWASP Top 10:2021 [4], and OWASP has since released a revised Top 10:2025 [31] with renumbered and, in some cases, redefined categories, to which the current rule set has not yet been remapped.

Future work will investigate evaluation on production Kubernetes traces from multi-node AKS/GKE clusters, formal statistical comparison against HPA and Keptn on shared infrastructure, extending the federated RL protocol for multi-cluster telemetry aggregation, generalizing the OWASP detection rules beyond name-based heuristics toward data-flow/taint analysis, empirically calibrating the compliance-scoring severity weights, remapping the OWASP rule set to the Top 10:2025 taxonomy [31], integrating live vulnerability feeds such as NVD [32], OSV [33], and GitHub Dependabot [34] in place of the current curated CVE list, and evaluating the Shift-Left engine on a larger sample of public open-source repository PRs.

The complete codebase is publicly available at <https://github.com/AadiHaldar/Agentic-AI-Self-Healing-Cloud-Infrastructure> and deployed at <https://pr-review-agent.wonderfulflower-41d6d2a5.eastasia.azurecontainerapps.io>.

REFERENCES

- [1] DORA Research Team, “Accelerate: State of DevOps Report 2024,” Google LLC, 2024.
- [2] E. Kim, J. Humble, P. Debois, and J. Willis, “The DevOps Handbook,” IT Revolution Press, 2016.

- [3] N. Forsgren, J. Humble, and G. Kim, "Accelerate: The Science of Lean Software and DevOps," IT Revolution Press, 2018.
- [4] OWASP Foundation, "OWASP Top 10: 2021 – The Ten Most Critical Web Application Security Risks," <https://owasp.org/Top10/>, 2021.
- [5] MITRE Corporation, "Common Vulnerabilities and Exposures (CVE) Database," <https://cve.mitre.org>, 2024.
- [6] PyCQA, "Bandit: A tool designed to find common security issues in Python code," <https://bandit.readthedocs.io>, 2024.
- [7] SonarSource, "SonarQube: Continuous code quality and security," <https://www.sonarqube.org>, 2024.
- [8] Kubernetes Documentation, "Horizontal Pod Autoscaler," <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>, 2024.
- [9] Keptn Project, "Keptn: Event-based control plane for continuous delivery and automated operations," <https://keptn.sh>, 2023.
- [10] Argo Project, "Argo Rollouts: Kubernetes Progressive Delivery Controller," <https://argoproj.github.io/rollouts/>, 2024.
- [11] T. Wang et al., "Multi-Agent Reinforcement Learning for Dynamic Resource Management in Cloud Computing," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 891–905, 2023.
- [12] H. Li, Z. Wang, and J. Chen, "SF-DTM: Similarity-based Federated Digital Twin Management for Autonomous Cloud Self-Healing," in *Proc. IEEE CLOUD*, 2024, pp. 234–242.
- [13] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. 8th IEEE ICDM*, 2008, pp. 413–422.
- [14] Q. Lin et al., "AIOps: Real-World Challenges and Research Innovations," in *Proc. ICSE-SEIP*, 2021, pp. 4–15.
- [15] Z. Li, M. Fan, and J. Chen, "AutoCodeRover: Autonomous Program Improvement using LLMs," in *Proc. ISSTA*, 2024, pp. 1–12.
- [16] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Proc. NeurIPS*, 2017, pp. 4765–4774.
- [17] Google DeepMind, "Gemini 2.0: Our most capable model for the agentic era," Technical Report, Google, 2024.
- [18] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. ICLR*, 2023.
- [19] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. Wiley-Interscience, 1975.
- [20] B. Zeigler, H. Praehofer, and T. G. Kim, *Theory of Modeling and Simulation*, 2nd ed. Academic Press, 2000.
- [21] K. Fall and K. Varadhan, "The ns Manual," The VINT Project, UC Berkeley, 2011.
- [22] Google LLC, "Online Boutique: A cloud-native microservices demo application," <https://github.com/GoogleCloudPlatform/microservices-demo>, 2024.
- [23] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, "Site Reliability Engineering: How Google Runs Production Systems," O'Reilly Media, 2016.
- [24] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, "Chaos Engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.
- [25] J. Humble and D. Farley, "Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation," Addison-Wesley, 2010.
- [26] M. Fowler, "BlueGreenDeployment," <https://martinfowler.com/bliki/BlueGreenDeployment.html>, 2010.
- [27] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.
- [28] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proc. AISTATS*, 2017, pp. 1273–1282.
- [29] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying Density-Based Local Outliers," in *Proc. ACM SIGMOD*, 2000, pp. 93–104.
- [30] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You? Explaining the Predictions of Any Classifier," in *Proc. ACM SIGKDD*, 2016, pp. 1135–1144.
- [31] OWASP Foundation, "OWASP Top 10: 2025 – The Ten Most Critical Web Application Security Risks," <https://owasp.org/Top10/>, 2025.
- [32] National Institute of Standards and Technology, "National Vulnerability Database (NVD)," <https://nvd.nist.gov>, 2024.
- [33] Google / OSV.dev, "OSV: Open Source Vulnerabilities Database," <https://osv.dev>, 2024.
- [34] GitHub, Inc., "Dependabot: Automated Dependency Updates," <https://docs.github.com/en/code-security/dependabot>, 2024.
- [35] Snyk Ltd., "Snyk Open Source: Software Composition Analysis," <https://snyk.io>, 2024.
- [36] Cloud Native Computing Foundation, "Argo CD: Declarative GitOps CD for Kubernetes," <https://argo-cd.readthedocs.io>, 2024.
- [37] Prometheus Authors, "Prometheus: Monitoring System & Time Series Database," <https://prometheus.io>, 2024.